Supply Chain Visibility Platform Personal

Study Notes

Prepared By: Simbarashe Chindanga August 14, 2026

About This Document: This is a personal study guide that documents the complete build process of the Supply Chain Visibility Platform. Each phase is broken down with file names, code explanations, and commands used. Use this to revisit, understand, and modify the project later.

Contents

1 Introduction **3**
1.1 Project Overview . . . 3
1.2 What We Built . . . 3

2 Project Structure **4**

3 Phase 1: Project Setup **5**
3.1 What We Did . . . 5
3.2 Files Created . . . 5
3.2.1 backend/package.json . . . 5
3.2.2 frontend/package.json . . . 5
3.3 Commands Used . . . 6
3.4 Key Takeaways . . . 6

4 Phase 2: Backend API **7**
4.1 What We Did . . . 7
4.2 Files Created/Updated . . . 7
4.2.1 backend/.env . . . 7
4.2.2 backend/server.js . . . 7
4.2.3 backend/src/routes/testRoutes.js . . . 8
4.3 Commands Used . . . 8
4.4 How to Test . . . 8
4.5 Key Takeaways . . . 8

5 Phase 3: Frontend Dashboard **10**
5.1 What We Did . . . 10
5.2 Files Created/Updated . . . 10
5.2.1 frontend/src/App.js (Shipment Data) . . . 10
5.2.2 frontend/src/App.js (Tracking Function) . . . 10
5.3 Key Shipment Fields . . . 11
5.4 How It Works . . . 11
5.5 Key Takeaways . . . 11

6 Phase 4: Authentication **12**
6.1 What We Did . . . 12
6.2 Files Created/Updated . . . 12
6.2.1 backend/src/utils/userUtils.js . . . 12
6.2.2 backend/src/routes/authRoutes.js . . . 13
6.2.3 backend/users.json . . . 14
6.3 How Authentication Works . . . 15
6.4 Key Takeaways . . . 15

7 Phase 5: Browser Notifications **16**
7.1 What We Did . . . 16
7.2 Files Created/Updated . . . 16
7.2.1 frontend/src/utils/notifications.js . . . 16
7.3 How It Works . . . 17
7.4 Key Takeaways . . . 17

8 Phase 6: Analytics Dashboard **18**
8.1 What We Did . . . 18
8.2 Files Created/Updated . . . 18
8.2.1 frontend/src/pages/Analytics.js . . . 18
8.3 How It Works . . . 19
8.4 Key Takeaways . . . 19

9 Phase 7: 50 Shipments Expansion **20**
9.1 What We Did . . . 20
9.2 New Routes Added . . . 20
9.3 Key Takeaways . . . 20

10 Phase 8: Final Polish **21**
10.1 What We Did . . . 21
10.2 Key Fixes . . . 21
10.2.1 Case-Insensitive Search . . . 21
10.2.2 Typing Issue Fix . . . 21
10.2.3 Responsive Design . . . 21
10.3 Key Takeaways . . . 21

11 Commands Reference **22**
11.1 Start Backend . . . 22
11.2 Start Frontend . . . 22
11.3 Stop Servers . . . 22
11.4 Kill All Node Processes . . . 22
11.5 Test Endpoints . . . 22
11.6 Login Credentials . . . 22
11.7 Test Shipments . . . 22

12 Conclusion **23**
12.1 What Was Built . . . 23
12.2 Skills Demonstrated . . . 23
12.3 Next Steps for Learning . . . 23

1 Introduction

This document serves as a complete study guide for the **Supply Chain Visibility Platform** — a full-stack web application for tracking shipments across Zimbabwe.

1.1 Project Overview

- **Name:** Supply Chain Visibility Platform
- **Location:** Zimbabwe
- **Goal:** Track 50+ shipments across all provinces with maps, analytics, and notifications
- **Tech Stack:** React + Node.js + Express + Leaflet + Recharts

1.2 What We Built

- 50 Zimbabwean shipments covering all 10 provinces
- Interactive map with route lines and markers
- Login/Register system with JWT authentication
- Browser notifications for status changes
- Analytics dashboard with charts and stats
- Border crossing tracking (Beitbridge, Chirundu, Nyamapanda, Plumtree)

2 Project Structure

```
03_supply_chain_visibility_platform/

 backend/
    src/
       models/
          User.js
       routes/
          testRoutes.js
          authRoutes.js
       utils/
           userUtils.js
    scripts/
       seedData.js
    .env
    package.json
    server.js
    users.json

 frontend/
    public/
       index.html
    src/
       components/
          ShipmentMap.js
          Navbar.js
       context/
          AuthContext.js
       pages/
          Login.js
          Register.js
          Analytics.js
       utils/
          notifications.js
       App.js
       App.css
       index.js
       index.css
    package.json
    package-lock.json

 database/
 docs/
 tests/
```

3 Phase 1: Project Setup

3.1 What We Did

Created the folder structure and installed all dependencies.

3.2 Files Created

3.2.1 backend/package.json

```
{
  "name": "backend",
  "version": "1.0.0",
  "description": "Supply Chain Visibility Platform - Backend API",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "cors": "^2.8.5",
    "dotenv": "^16.0.3",
    "jsonwebtoken": "^9.0.0",
    "bcryptjs": "^2.4.3",
    "socket.io": "^4.5.4"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  }
}
```

3.2.2 frontend/package.json

```
{
  "name": "frontend",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "axios": "^1.6.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.20.0",
    "react-scripts": "5.0.1",
    "socket.io-client": "^4.5.4",
    "leaflet": "^1.9.4",
    "react-leaflet": "^4.2.1",
    "recharts": "^2.10.0"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "test": "react-scripts test",
```

```
    "eject": "react-scripts eject"
  }
}
```

3.3 Commands Used

```
# Backend
cd backend
npm init -y
npm install express cors dotenv nodemon jsonwebtoken bcryptjs socket.io

# Frontend
cd frontend
npm init -y
npm install react react-dom react-router-dom axios react-scripts
npm install leaflet react-leaflet recharts socket.io-client
```

3.4 Key Takeaways

- Express handles the backend API
- React handles the frontend UI
- Nodemon auto-restarts backend on changes
- React Scripts handles frontend build and development

4 Phase 2: Backend API

4.1 What We Did

Built the Express server with test routes to verify the API is working.

4.2 Files Created/Updated

4.2.1 backend/.env

```
PORT=5000
JWT_SECRET=your_super_secret_key_change_this_later
NODE_ENV=development
FRONTEND_URL=http://localhost:3000
```

4.2.2 backend/server.js

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const http = require('http');
const { Server } = require('socket.io');

dotenv.config();

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
    cors: {
        origin: "http://localhost:3000",
        methods: ["GET", "POST", "PUT", "DELETE"]
    }
});

app.use(cors());
app.use(express.json());

app.get('/api/test', (req, res) => {
    res.json({
        success: true,
        message: 'Supply Chain Visibility API is working!',
        timestamp: new Date().toISOString()
    });
});

app.get('/api/health', (req, res) => {
    res.json({
        status: 'healthy',
        uptime: process.uptime(),
        timestamp: new Date().toISOString()
    });
});
```

```
io.on('connection', (socket) => {
    console.log(' New client connected:', socket.id);
    socket.on('disconnect', () => {
        console.log(' Client disconnected:', socket.id);
    });
});

const PORT = process.env.PORT || 5000;
server.listen(PORT, () => {
    console.log(` Server running on http://localhost:${PORT}`);
});
```

4.2.3 backend/src/routes/testRoutes.js

```
const express = require('express');
const router = express.Router();

router.get('/test', (req, res) => {
    res.json({
        success: true,
        message: 'API is working!',
        endpoints: {
            'GET /api/test': 'Test this endpoint',
            'GET /api/health': 'Health check'
        }
    });
});

router.get('/health', (req, res) => {
    res.json({
        status: 'healthy',
        uptime: process.uptime()
    });
});

module.exports = router;
```

4.3 Commands Used

```
cd backend
npm run dev
```

4.4 How to Test

1. Start backend: `cd backend && npm run dev`
2. Visit: `http://localhost:5000/api/test`
3. You should see a JSON response

4.5 Key Takeaways

- Express creates the server

- CORS allows frontend to communicate with backend
- Socket.io is ready for real-time updates
- Dotenv loads environment variables from `.env`
- Test routes verify the API is working

5 Phase 3: Frontend Dashboard

5.1 What We Did

Built the React dashboard with 50 Zimbabwean shipments and search functionality.

5.2 Files Created/Updated

5.2.1 frontend/src/App.js (Shipment Data)

```
const zimbabweRoutes = {
    'SHIP-001': {
        origin: 'Harare, Zimbabwe',
        destination: 'Bulawayo, Zimbabwe',
        route: 'Harare -> Bulawayo',
        distance: '440 km',
        eta: '2026-08-07 16:30',
        status: 'In Transit',
        location: { lat: -19.0154, lng: 29.1549 }
    }
    // ... 49 more shipments
};
```

5.2.2 frontend/src/App.js (Tracking Function)

```
const trackShipment = async (e) => {
    e.preventDefault();
    if (!shipmentId.trim()) return;

    setLoadingShipment(true);
    try {
        const searchId = shipmentId.trim().toUpperCase();
        const shipment = zimbabweRoutes[searchId];

        if (shipment) {
            const shipmentData = {
                id: searchId,
                ...shipment,
                lastUpdate: new Date().toISOString(),
            };
            setTrackingData(shipmentData);
        } else {
            alert('Shipment not found. Try SHIP-001 to SHIP-050');
            setTrackingData(null);
        }
    } catch (error) {
        console.error('Error tracking shipment:', error);
    }
    setLoadingShipment(false);
};
```

Field	Description
origin	Starting city (e.g., Harare, Zimbabwe)
destination	Ending city (e.g., Bulawayo, Zimbabwe)
route	Display name (e.g., Harare -> Bulawayo)
distance	Distance in km (e.g., 440 km)
eta	Estimated arrival time
status	Current status
location	GPS coordinates {lat, lng}

5.3 Key Shipment Fields

5.4 How It Works

1. User enters a shipment ID (e.g., SHIP-001)
2. `trackShipment()` converts input to uppercase
3. Searches for the ID in `zimbabweRoutes`
4. If found, displays shipment details
5. If not found, shows an error message

5.5 Key Takeaways

- State stores shipment data and loading status
- Case-insensitive search improves user experience
- Zimbabwe-specific routes make it locally relevant
- 50 shipments cover all 10 provinces

6 Phase 4: Authentication

6.1 What We Did

Added JWT-based login and registration using JSON file storage.

6.2 Files Created/Updated

6.2.1 backend/src/utils/userUtils.js

```
const fs = require('fs');
const path = require('path');
const bcrypt = require('bcryptjs');

const usersFilePath = path.join(__dirname, '../../users.json');

function readUsers() {
    try {
        const data = fs.readFileSync(usersFilePath, 'utf8');
        return JSON.parse(data);
    } catch (error) {
        return [];
    }
}

function writeUsers(users) {
    fs.writeFileSync(usersFilePath, JSON.stringify(users, null, 2));
}

async function createUser(userData) {
    const users = readUsers();

    if (users.find(u => u.username === userData.username)) {
        throw new Error('Username already exists');
    }

    const salt = await bcrypt.genSalt(10);
    const hashedPassword = await bcrypt.hash(userData.password, salt);

    const newUser = {
        id: Date.now().toString(),
        ...userData,
        password: hashedPassword,
        isActive: true,
        createdAt: new Date().toISOString()
    };

    users.push(newUser);
    writeUsers(users);

    const { password, ...userWithoutPassword } = newUser;
    return userWithoutPassword;
```

```
}

async function validateUser(usernameOrEmail, password) {
    const user = readUsers().find(u =>
        u.username === usernameOrEmail ||
        u.email === usernameOrEmail
    );
    if (!user) return null;

    const isValid = await bcrypt.compare(password, user.password);
    if (!isValid) return null;

    const { password: _, ...userWithoutPassword } = user;
    return userWithoutPassword;
}

module.exports = { readUsers, createUser, validateUser };
```

6.2.2 backend/src/routes/authRoutes.js

```
const express = require('express');
const router = express.Router();
const jwt = require('jsonwebtoken');
const { createUser, validateUser } = require('../utils/userUtils');

router.post('/register', async (req, res) => {
    try {
        const { username, email, password, fullName, role } = req.body;

        if (!username || !email || !password || !fullName) {
            return res.status(400).json({
                success: false,
                message: 'Please provide all required fields'
            });
        }

        const newUser = await createUser({
            username,
            email,
            password,
            fullName,
            role: role || 'customer'
        });

        const token = jwt.sign(
            { id: newUser.id, username: newUser.username, role: newUser.role },
            process.env.JWT_SECRET,
            { expiresIn: '7d' }
        );

        res.status(201).json({ success: true, token, user: newUser });
    } catch (error) {
```

```
        res.status(400).json({ success: false, message: error.message });
    }
});

router.post('/login', async (req, res) => {
    try {
        const { username, password } = req.body;

        if (!username || !password) {
            return res.status(400).json({
                success: false,
                message: 'Please provide username/email and password'
            });
        }

        const user = await validateUser(username, password);
        if (!user) {
            return res.status(401).json({
                success: false,
                message: 'Invalid username/email or password'
            });
        }

        const token = jwt.sign(
            { id: user.id, username: user.username, role: user.role },
            process.env.JWT_SECRET,
            { expiresIn: '7d' }
        );

        res.json({ success: true, token, user });
    } catch (error) {
        res.status(500).json({ success: false, message: error.message });
    }
});

module.exports = router;
```

6.2.3 backend/users.json

```
[
    {
        "id": "1",
        "username": "admin",
        "email": "admin@supplychain.com",
        "password": "$2a$10$XrQeY5zQxW5zQxW5zQxW5e",
        "role": "admin",
        "fullName": "System Administrator",
        "phoneNumber": "+263771234567",
        "company": "Supply Chain Zimbabwe",
        "isActive": true,
        "createdAt": "2026-08-11T12:00:00.000Z"
    },
```

```
    {
        "id": "2",
        "username": "driver1",
        "email": "driver1@supplychain.com",
        "password": "$2a$10$XrQeY5zQxW5zQxW5zQxW5e",
        "role": "driver",
        "fullName": "Tendai Moyo",
        "phoneNumber": "+263771234568",
        "company": "Harare Logistics",
        "isActive": true,
        "createdAt": "2026-08-11T12:00:00.000Z"
    },
    {
        "id": "3",
        "username": "customer1",
        "email": "customer1@supplychain.com",
        "password": "$2a$10$XrQeY5zQxW5zQxW5zQxW5e",
        "role": "customer",
        "fullName": "Simbarashe Chindanga",
        "phoneNumber": "+263771234569",
        "company": "Zimbabwe Traders",
        "isActive": true,
        "createdAt": "2026-08-11T12:00:00.000Z"
    }
]
```


6.3 How Authentication Works

1. User submits login form
2. Backend validates credentials from `users.json`
3. On success, backend returns a JWT token
4. Frontend stores token in `localStorage`
5. Token is sent in headers for all protected requests

6.4 Key Takeaways

- JWT provides secure token-based authentication
- bcryptjs hashes passwords before storing
- JSON file stores users (can be replaced with database)
- AuthContext manages auth state across the app

7 Phase 5: Browser Notifications

7.1 What We Did

Added browser notifications for shipment status changes.

7.2 Files Created/Updated

7.2.1 frontend/src/utils/notifications.js

```
export async function requestNotificationPermission() {
    if (!('Notification' in window)) return false;

    if (Notification.permission === 'granted') return true;
    if (Notification.permission === 'denied') return false;

    const permission = await Notification.requestPermission();
    return permission === 'granted';
}

export function sendNotification(title, options = {}) {
    if (Notification.permission !== 'granted') return;

    const notification = new Notification(title, {
        icon: '',
        ...options
    });

    setTimeout(() => notification.close(), 5000);
}

export function sendShipmentNotification(shipment) {
    let title = ' Shipment Update';
    let body = `${shipment.id}: ${shipment.status}`;

    if (shipment.status.includes('Delivered')) {
        title = ' Shipment Delivered!';
        body = `${shipment.id} delivered successfully`;
    } else if (shipment.status.includes('Delayed')) {
        title = ' Shipment Delayed';
        body = `${shipment.id} is delayed`;
    } else if (shipment.status.includes('Border')) {
        title = ' Shipment at Border';
        body = `${shipment.id} is at border crossing`;
    } else if (shipment.status.includes('In Transit')) {
        title = ' Shipment In Transit';
        body = `${shipment.id} is on its way`;
    }

    sendNotification(title, { body });
}
```

7.3 How It Works

1. User grants notification permission
2. When a shipment is tracked, notification is sent
3. Different statuses trigger different notification types
4. Notifications auto-close after 5 seconds

7.4 Key Takeaways

- Web Notification API is built into modern browsers
- Permission is requested on login
- Status-based notifications improve user experience

8 Phase 6: Analytics Dashboard

8.1 What We Did

Added charts and stats using Recharts.

8.2 Files Created/Updated

8.2.1 frontend/src/pages/Analytics.js

```
import React, { useState, useEffect } from 'react';
import { BarChart, Bar, PieChart, Pie, Cell, XAxis, YAxis, Tooltip, Legend, ResponsiveContain

function Analytics({ shipments }) {
    const [stats, setStats] = useState({
        total: 0,
        byStatus: [],
        byRoute: [],
        topRoutes: [],
        deliveryRate: 0
    });

    useEffect(() => {
        if (shipments && Object.keys(shipments).length > 0) {
            calculateStats(shipments);
        }
    }, [shipments]);

    const calculateStats = (data) => {
        const shipmentList = Object.values(data);
        const total = shipmentList.length;

        const statusCount = {};
        const routeCount = {};
        let deliveredCount = 0;

        shipmentList.forEach(ship => {
            const status = ship.status;
            statusCount[status] = (statusCount[status] || 0) + 1;
            if (status.includes('Delivered')) deliveredCount++;

            const route = ship.route;
            routeCount[route] = (routeCount[route] || 0) + 1;
        });

        const statusData = Object.keys(statusCount).map(key => ({
            name: key,
            value: statusCount[key]
        }));

        const routeData = Object.keys(routeCount)
            .map(key => ({
```

```
                name: key.length > 20 ? key.substring(0, 20) + '...' : key,
                fullName: key,
                value: routeCount[key]
            }))
            .sort((a, b) => b.value - a.value)
            .slice(0, 10);

        const deliveryRate = total > 0 ? Math.round((deliveredCount / total) * 100) : 0;

        setStats({
            total,
            byStatus: statusData,
            byRoute: routeData,
            topRoutes: routeData.slice(0, 5),
            deliveryRate
        });
    };

    return (
        <div>
            <h2> Supply Chain Analytics</h2>
            <div>
                <h3>Total Shipments: {stats.total}</h3>
                <h3>Delivery Rate: {stats.deliveryRate}%</h3>
            </div>
        </div>
    );
}

export default Analytics;
```

8.3 How It Works

1. Analytics receives `zimbabweRoutes` as a prop
2. `calculateStats()` loops through all 50 shipments
3. Counts statuses and routes
4. Calculates delivery rate
5. Recharts renders pie and bar charts

8.4 Key Takeaways

- Recharts is a React charting library
- Pie Chart shows status distribution
- Bar Chart shows top routes
- Stats cards show key metrics

9 Phase 7: 50 Shipments Expansion

9.1 What We Did

Expanded from 20 to 50 shipments covering all 10 provinces.

9.2 New Routes Added

Province	**Routes**
Harare	Harare -> Bulawayo, Mutare, Beitbridge, Victoria Falls, Chirundu, Gweru, Masvingo,
Bulawayo	Bulawayo -> Harare, Victoria Falls, Beitbridge, Gweru, Plumtree
Manicaland	Mutare -> Harare, Nyamapanda, Masvingo, Chimanimani
Matabeleland South	Beitbridge -> Harare, Bulawayo, Masvingo
Matabeleland North	Victoria Falls -> Harare, Bulawayo, Kariba
Midlands	Gweru -> Harare, Bulawayo, Kwekwe
Other	Masvingo, Chiredzi, Kariba, Plumtree, Nyamapanda routes

9.3 Key Takeaways

- 50 shipments cover all 10 provinces
- 4 border crossings are monitored
- 5,000+ km of routes are tracked

10 Phase 8: Final Polish

10.1 What We Did

Fixed bugs, improved UI, and added small features.

10.2 Key Fixes

10.2.1 Case-Insensitive Search

```
const searchId = shipmentId.trim().toUpperCase();
const shipment = zimbabweRoutes[searchId];
```

10.2.2 Typing Issue Fix

```
<input
    type="text"
    value={shipmentId}
    onChange={(e) => setShipmentId(e.target.value)}
    autoFocus
/>
```

10.2.3 Responsive Design

```
@media (max-width: 600px) {
    .details-grid {
        grid-template-columns: 1fr;
    }
    .tracking-form {
        flex-direction: column;
    }
}
```

10.3 Key Takeaways

- autoFocus keeps the input focused
- toUpperCase() makes search case-insensitive
- Media queries make the app responsive

11 Commands Reference

11.1 Start Backend

```
cd backend
npm run dev
```

11.2 Start Frontend

```
cd frontend
npm start
```

11.3 Stop Servers

Press `Ctrl + C` in each terminal.

11.4 Kill All Node Processes

```
taskkill /F /IM node.exe
```

11.5 Test Endpoints

```
http://localhost:5000/api/test
http://localhost:5000/api/health
```

11.6 Login Credentials

Username	Password	Role
admin	password123	Admin
driver1	password123	Driver
customer1	password123	Customer

11.7 Test Shipments

ID	Route	Status
SHIP-001	Harare -> Bulawayo	In Transit
SHIP-003	Harare -> Beitbridge	In Transit
SHIP-004	Harare -> Victoria Falls	In Transit
SHIP-010	Beitbridge -> Harare	At Border
SHIP-017	Bulawayo -> Victoria Falls	Delayed

12 Conclusion

12.1 What Was Built

- Full-stack supply chain tracking platform
- 50 Zimbabwean shipments across all provinces
- Interactive mapping with Leaflet
- JWT-based authentication
- Browser notifications
- Analytics dashboard with Recharts

12.2 Skills Demonstrated

- React (Hooks, State Management, Routing)
- Node.js + Express (REST APIs, WebSockets)
- Authentication (JWT, bcrypt)
- Data Visualization (Recharts)
- Mapping (Leaflet)
- Security (Password Hashing, Token Auth)
- Full-Stack Development

12.3 Next Steps for Learning

- Add database (MongoDB or PostgreSQL)
- Deploy online (Vercel + Render)
- Add email/SMS notifications
- Build mobile app
- Add predictive analytics